# [Experiment, Analysis, and Benchmark]BEACON: A Benchmark for Efficient and Accurate Counting of Subgraphs


Xiangju Zhu
*The University of Hong Kong*
Hong Kong SAR, China
zxj0302@connect.hku.hk

Mohammad Matin Najafi
*Huawei Hong Kong Research Center*
Hong Kong SAR, China
matin.najafi97@huawei.com

Chrysanthi Kosyfaki
*The Hong Kong University of Science and Technology*
Hong Kong SAR, China
ckosyfaki@cse.ust.hk

Xiaodong Li
*Xiamen University*
Xiamen, China
xdli@xmu.edu.cn

Reynold Cheng
*The University of Hong Kong*
Hong Kong SAR, China
ckcheng@cs.hku.hk

Laks V.S. Lakshmanan
*University of British Columbia*
Vancouver, Canada
laks@cs.ubc.ca



***Abstract*—Subgraph counting, the task of determining the number of instances of a query pattern within a large graph, plays an important role in many real-world applications, from analyzing financial networks and transportation systems to understanding biological interactions. Although this problem has been widely explored, ranging from efficient algorithmic (AL) solutions to more recently developed machine learning (ML) approaches, systematic comparative insights, particularly direct comparisons between AL and ML methods, remain elusive. This gap stems from the absence of a unified evaluation framework, standardized datasets, and accessible ground truths, all of which hinder systematic analysis and fair benchmarking. To overcome these barriers, we introduce *BEACON*: a benchmark for efficient and accurate counting of subgraphs to evaluate both AL and ML-based subgraph counting methods. BEACON provides a standardized dataset with verified ground truths, an integrated evaluation environment, and a public leaderboard, enabling reproducible and transparent comparisons across diverse approaches. Our extensive experiments show that while AL methods perform more efficiently in counting subgraphs on very large graphs, they struggle with complex patterns (e.g. those exceeding six nodes). In contrast, ML methods are capable of handling larger patterns but demand massive graph data inputs and often yield suboptimal accuracy on complex graphs. These insights not only highlight the unique strengths and limitations of each approach but also pave the way for future advancements in subgraph counting techniques. Overall, BEACON represents a significant step towards unifying and accelerating research in subgraph counting, encouraging innovative solutions and fostering a deeper understanding of the trade-offs between algorithmic and machine learning paradigms.**



***Index Terms*—Subgraph Counting, Graph Neural Networks, GNN, Benchmark, Graph Mining, Motif Counting**


## I. Introduction

Subgraph counting is a fundamental problem in the area of graph analytics, where the objective is to count the number of subgraphs in a large graph $G = (V, E)$ that are isomorphic to a given query graph $q = (V_q, E_q)$ [1]. Formally, the goal is to compute $\mathcal{N}_q(G)$, the number of occurrences of $q$ within $G$.

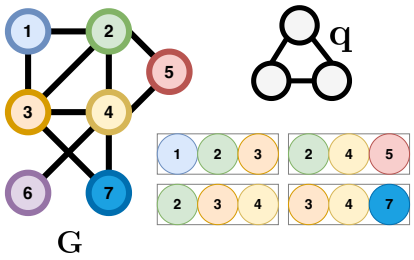



Fig. 1. An example input graph $G$, a query pattern $q$, and the 4 corresponding instances of $q$ in $G$, yielding $\mathcal{N}_q(G) = 4$.

Subgraph counting plays a vital role in many domains [2]–[16]. In network science, detecting frequently occurring patterns such as cliques (fully connected subgraphs) or recurring motifs can reveal community structures and interaction dynamics within social networks [17]. In biological networks, counting subgraphs such as motifs is crucial in revealing functional modules or biologically significant interactions [18]–[20]. Furthermore, in financial and transaction networks, the detection and counting of specific subgraph patterns can expose anomalous transaction behaviors, which may signal fraudulent activities like money laundering [21]. These applications underscore the important role of subgraph counting for extracting deep insights about complex and interconnected systems. Fig. 1 illustrates the pattern $q$ (a triangle), and its instances in graph $G$. Due the NP-completeness of the underlying subgraph isomorphism, subgraph counting is computationally challenging [22], [23]. This problem intensifies when $G$ is huge, or when $q$ is large. To address these issues, two classes of solutions were studied in the literature: Algorithmic (AL) and Machine Learning (ML)-based, as discussed below.

Algorithmic methods include exact and approximate techniques. State-of-the-art exact methods leverage advanced com-

binatorial strategies and specialized hand-crafted formulas adjusted to specific subgraph patterns (e.g. ESCAPE [24] and EVOKE [25]), which can compute counts efficiently when such formulas exist. However, deriving these formulas is inherently challenging and they usually do not support larger or more complex subgraphs. Approximate AL methods (e.g., MOTIVO [26]) aims to trade accuracy for efficiency. Their performance can degrade significantly as both $G$ and $q$ become more complex.

Machine Learning-based approaches have emerged as a promising alternative [27]–[34] to traditional algorithmic methods for subgraph counting. These methods use graph patterns to train a model. The trained model can then produce subgraph counts, even for large and complex patterns. However, there is a lack of a unified framework and curated datasets for ML-based counting, making standardized evaluation against AL strategies difficult. The performance of these models is also sensitive to variations in software environments, hardware configurations, and training procedures, raising reproducibility concerns. Moreover, we observe that in existing works in ML approaches, only few AL methods, which may not be state of the art (SOTA), were used for comparisons. Without a comprehensive and systematic comparison, it is not clear *when* and *how* ML approaches fare better than AL methods. This makes it difficult to realize the strengths and weaknesses of existing subgraph counting algorithms, or design better ones.

*a) The BEACON benchmark:* To address the limitations and open questions of current subgraph counting approaches, we propose *BEACON*, a Benchmark for Efficient and Accuracy COuNting. This is the first unified framework that enables comparisons among AL and ML-based methods across various types of graphs and query patterns. It also enhances reproducibility through standardized datasets, detailed protocols, and containerized environments. It can also be useful to the research community by enlightening the trade-offs among scalability, accuracy, and runtime that are made by the various methods. BEACON facilitates a deeper understanding of existing methodologies and supports the development of next-generation methods that effectively harness both algorithmic and ML techniques. Specifically, our contributions are:

- **A Unified Benchmark Framework.** We introduce *BEACON*, the first standardized benchmark, to our knowledge, that enables fair comparison of both AL and ML–based subgraph counting techniques. BEACON integrates datasets, evaluation pipelines, and containerized environments to ensure transparency and reproducibility.
- **Curated Datasets with Verified Ground Truth.** We construct a large-scale *Oracle Dataset* containing over 26K graphs from multiple domains, with exact ground-truth counts for all subgraphs up to five nodes (induced and non-induced, local and global). Using this, we further provide a *Benchmark Dataset* for controlled evaluation under diverse scenarios of size, density, and complexity.
- **Flexible Data Sampling Tool.** We release the *BEACON-Sampler*, a configurable tool (PyPI package) that allows researchers to generate custom benchmark subsets by specifying constraints such as graph size, node degree, clustering coefficient, or subgraph count range. This facilitates targeted experiments and stress testing.
- **Comprehensive Evaluation and Insights.** Through extensive experiments on real-world networks, BEACON systematically benchmarks exact, approximate, and ML-based methods. Our evaluation uncovers critical trade-offs in *accuracy, scalability, and runtime efficiency*, highlighting when ML methods excel over AL approaches and where their weaknesses lie.
- **Reproducibility and Community Resources.** To ensure extensibility, BEACON provides: a containerized environment (via Docker) for plug-and-play evaluation of methods; a public leaderboard that tracks state-of-the-art performance, fostering ongoing collaboration; and an open-source pipeline that welcomes community contributions of new datasets, algorithms, and improvements.

To summarize, we propose a robust ecosystem for subgraph counting research that delivers in-depth experimental insights while providing an easy-to-use, reproducible platform that accelerates future developments in this challenging domain.

*b) Roadmap:* The rest of the paper is organized as follows. In Section II, we discuss the problem definitions and notations used in the paper. Section III reviews related work on AL- and ML-based subgraph counting approaches. Section IV details our proposed framework. Section V presents experimental evaluations on diverse real-world networks and insightful analysis. Section VI concludes.

## II. Notations and Definitions

In this section, we introduce the primary notations and definitions used throughout the paper.

**Definition 1** (Graph)**.** *A graph $G$ is defined as an ordered pair $(V, E)$, where $V$ is a finite set of nodes and $E \subseteq \{(u, v) \mid u, v \in V, u \neq v\}$ is a set of edges. Unless stated otherwise, we assume that $G$ is an undirected, simple graph: there are no self-loops (edges from a node to itself) and no parallel edges between the same pair of nodes. We denote the number of nodes by $n = |V|$ and the number of edges by $m = |E|$.*

A *subgraph* $H$ of $G$ is any graph whose node set and edge set are subsets of those in $G$. Formally, $H = (V_H, E_H)$ is a subgraph of $G = (V, E)$ if and only if $V_H \subseteq V$ and $E_H \subseteq E$. We are often interested in patterns that have a predetermined shape or size (e.g. patterns with 4 nodes).

**Definition 2** (Subgraph Counting)**.** *Let $F$ be a target subgraph, and let $G$ be a graph. We define*

$$\mathcal{N}_F(G) \;=\; \big|\{\, H \subseteq G : H \cong F \}\big|,$$

*where $H \cong F$ denotes that $H$ is isomorphic to $F$. The* subgraph counting *problem is to compute $\mathcal{N}_F(G)$. Two subgraph instances are considered distinct if they differ in at least one node or edge.*

Subgraph counting can be performed at multiple levels of granularity. The Definition 2 counts the total number of

isomorphisms and therefore is referred to as global subgraph counting.

**Definition 3** (Local Subgraph Counting)**.** *For each node $v \in V$, count the number of subgraphs that include $v$ and are isomorphic to $F$. Formally,*

$$\mathcal{N}_F(v; G) \;=\; \big|\{\, H \subseteq G : v \in V(H),\; H \cong F\}\big|.$$

*The result is a vector $\big(\mathcal{N}_F(v_1; G), \ldots, \mathcal{N}_F(v_n; G)\big)$, one entry per node.*

*a) Subgraph-Centric vs. Network-Centric Methods:* Algorithms for subgraph counting are often categorized based on their input and output:

**Definition 4** (Subgraph-Centric Method)**.** *The algorithm is given a specific target subgraph $F$ as input and returns $\mathcal{N}_F(G)$. Examples include exact counting or estimation of a designated 4-node pattern with a particular connectivity structure.*

**Definition 5** (Network-Centric Method)**.** *The algorithm is given only the size $k$ of the subgraphs. The output is the number (or a vector of counts) of* all *non-isomorphic subgraphs of size $k$ present in $G$. Formally, one might compute*

$$\big(\mathcal{N}_{F_1}(G),\; \mathcal{N}_{F_2}(G),\; \ldots,\big),$$

*where each $F_i$ is a distinct unlabeled graph on $k$ nodes.*

*b) Subgraph Enumeration vs. Subgraph Counting:* Subgraph enumeration differs with subgraph counting in the level of detail they produce:

**Definition 6** (Subgraph Enumeration)**.** *Enumerate (i.e., explicitly list)* all *subgraph instances isomorphic to $F$ (or of size $k$). The enumeration procedure might return a set*

$$\{\, H_1, H_2, \ldots, H_r\} \quad \textit{where each } H_i \cong F,\; i = 1, \ldots, r.$$

*c) Induced vs. Non-Induced Subgraph Counting:* Let $S \subseteq V$ be a subset of nodes. The *induced subgraph* of $G$ on $S$ is the graph $G[S] = (S, E_S)$ where

$$E_S \;=\; \{\, (u, v) \in E : u \in S,\; v \in S\}.$$

In other words, $G[S]$ includes all edges in $E$ whose endpoints lie entirely in $S$.

**Definition 7** (Induced Subgraph Counting)**.** Induced subgraph counting *restricts attention to subgraphs $H \subseteq G$ that are induced by their node sets. Formally, if $V(H) = S$, then $E(H)$ must be $E_S$ as in $G[S]$.*

**Definition 8** (Non-Induced Subgraph Counting)**.** Non-induced subgraph counting *requires only that each subgraph $H \subseteq G$ matches a target pattern $F$ in node and edge configuration—but it is not necessary to include all edges among the chosen nodes. If $F$ has fewer edges than the complete graph on $V(F)$, the corresponding $H$ in $G$ may also omit those edges.*

*d) Q-error vs. Mean Absolute Error:* When estimating subgraph counts, it is crucial to quantify the discrepancy between an estimated count $\widehat{C}$ and the true count $C$. Two widely used metrics for subgraph count estimation are *Q-error* and *MAE*:

**Definition 9** (Q-error)**.** *Given $C > 0$ and $\widehat{C} > 0$, the Q-error is*

$$\textit{Q-error}(\widehat{C}, C) \;=\; \max\Big(\frac{\widehat{C}}{C},\; \frac{C}{\widehat{C}}\Big).$$

*A small Q-error implies that $\widehat{C}$ and $C$ are close relative to each other, emphasizing multiplicative deviations. To prevent division by zero, we use $C + 1$ and $\widehat{C} + 1$ in the numerator and denominator during evaluation.*

**Definition 10** (Mean Absolute Error (MAE))**.**

$$\textit{MAE} \;=\; \big|\widehat{C} - C\big|.$$

*MAE captures additive error, penalizing larger absolute discrepancies between the estimated and true values.*

*e) Scope of this paper:* This work focuses on *induced subgraph counting in homogeneous graphs* with particular emphasis on comparing state-of-the-art AL algorithms and recent ML methods for small patterns. We exclude approaches designed for labeled graphs (e.g., LSS [35]), as these methods rely heavily on node labels as features and typically degenerate when applied to unlabeled graphs.

## III. Related Work

The subgraph counting problem is well-studied in the literature [25], [35]–[42]. This problem can be challenging due to the size and complexity of the graph. As a result, there is a lot of interest in developing algorithms that use algorithmic or learning-based approaches to address these challenges. In this section, we summarize the most representative works.

*a) Exact/Approximate solutions:* Exact and approximate solutions have been extensively studied, particularly for their efficiency in counting subgraphs fast. These methods focus on reducing redundant computations and optimizing the search space to handle increasingly larger graphs. Pinar et al. [24] propose an algorithmic framework called ESCAPE to produce exact counts for all 5-vertex subgraphs. [25] studies the local subgraph counting problem. In the context of this work, the authors propose an algorithm called EVOKE. EVOKE introduces a novel approach by focusing on vertex orbits-specific configurations of vertices in subgraphs when counting 5-vertex subgraphs. By leveraging these orbits, EVOKE can capture structural patterns more effectively within the graphs. A recent work by Li and Yu [40] have also studied local subgraph counting queries. They propose a tree-decomposition-based algorithm that recursively breaks the graph into smaller subgraphs, applying symmetry-breaking techniques to reduce redundant counting. By incorporating symmetry-breaking rules, the algorithm can minimize the overhead of repeatedly counting identical substructures, resulting in a more efficient counting process.

MOTIVO [26] is an approximate solution that applies color coding to subgraph counting problem. The main contribution of MOTIVO is its focus on reducing both the time and memory consumption associated with traditional color coding methods. The authors introduce an adaptive graphlet sampling strategy that targets rare or extreme cases of graphlets, allowing it to efficiently count motifs in large-scale networks.

*b) Machine Learning-Based Approaches:* The rise of graph machine learning has driven significant interest in neural approaches for subgraph counting [43]–[45]. Early works focus on understanding the fundamental capabilities of graph neural networks (GNNs), while recent innovations have introduced specialized architectures to overcome theoretical and practical limitations.

The expressive power of standard GNNs for substructure counting was first formalized by [36], which proved that message-passing networks (MPNNs) and 1-Weisfeiler-Lehman (1-WL) equivalent models cannot count connected induced substructures with $\geq 3$ nodes. You et al. [37] proposed Identity-aware Graph Neural Network (ID-GNN), which extends MPNNs with identity coloring and heterogeneous message passing to surpass the 1-WL test. Their approach that includes a faster cycle-count–based variant achieved notable improvements on node, edge, and graph-level tasks. Subsequent works sought to enhance pattern recognition while maintaining efficiency. Zhao et al. [46] introduced GNN-AK, a framework that boosts the expressiveness of MPNNs by encoding rooted subgraphs with a base GNN. With the enhanced GNN-AK$^+$ and the efficient SubgraphDrop sampling, the approach achieves higher expressiveness than 1- and 2-WL. Maron et al. [47] propose graph networks provably as powerful as the k-WL test and introduce a simple second-order model with matrix multiplication that achieves 3-WL expressiveness, offering greater power than message passing GNNs while remaining scalable. For cycle counting, [48] proposed I$^2$-GNN, which augments subgraph MPNNs with dual-node identifiers to count 5- and 6-cycles, a task provably impossible for previous subgraph GNNs. Recent advances prioritize computational efficiency without sacrificing counting power. [49] proposed DeSCo, a canonical partitioning framework with heterogeneous message passing that reduces count prediction errors.

*c) Challenges in ML Approaches:* Despite these advances, critical challenges remain. First, the field lacks standardized benchmarks for fair ML-vs-algorithm comparisons, as noted by [37]. Reproducibility is restricted by undocumented hyperparameter sensitivities and hardware dependencies—issues exacerbated in memory intensive models. Furthermore, many studies focus on specific pattern families (e.g. cycles in [48]), making holistic performance assessments difficult. These gaps motivate our unified benchmarking framework in Section IV.

## IV. Proposed Framework

This section presents the details of our proposed benchmark framework, highlighting its modular structure, usage scenarios, and contribution opportunities. The framework is designed to facilitate the development, evaluation, and comparison of subgraph counting methods in a standardized and reproducible manner. Our benchmark is composed of three primary modules: (1) **Data**, (2) **Environment**, and (3) **Testing**. These modules are designed to cover the full spectrum of tasks required for subgraph counting method development and evaluation, from data preparation to testing and comparison. Importantly, our evaluation framework is built to assess both traditional algorithmic techniques and modern machine learning–based approaches, ensuring a comprehensive and unified analysis across diverse methodologies.

### A. Framework Overview

The core architecture of our framework is illustrated in Fig. 3(a). Each module serves a specific purpose:

*1) Data Module:* The data module is responsible for providing datasets required for training, validation, and testing subgraph counting methods. The module includes an Oracle Dataset and an interface, referred to as the BEACON-Sampler, which allows users to sample datasets based on specific user-defined constraints such as subgraph size, graph density, clustering coefficient, or the ground truth count range (e.g. the maximum local triangle count in graphs). The sampler ensures that datasets are selected in a way that aligns with the user's research objectives, depending on whether they are testing scalability, robustness, or generalizability.

*a) Oracle Dataset:* A key motivation for our Oracle Dataset is the absence of a large-scale dataset with ground truth subgraph counts in the community. To address this gap, we collected all graphs from the TUDataset [50] alongside graphs from the OGB dataset [51], which encompasses a variety of domains such as bioinformatics, social networks, and computer vision. For each graph, we computed the ground truth counts for all subgraphs with up to five nodes, considering both local and global frequencies as well as induced and non-induced configurations. We have made this extensive collection publicly available, and we refer to it as the *Oracle Dataset*. Our Oracle Dataset contains a total of 26,435 graphs drawn from multiple domains. In addition to the standard ground-truth subgraph counts, we augment the dataset with ID-constrained ground truth introduced by the DeSCo algorithm [49]. This approach reduces the complexity of ego-networks, particularly when high-degree nodes are assigned larger IDs. We believe this is a promising direction for improving the accuracy of local predictions, and we include this feature to encourage further research in this area. To further characterize each graph, we have computed many graph-level features including diameter, density, and clustering coefficient.

Fig. 2 illustrates the distribution of node and edge counts across domains, showing the 5th and 95th percentile trends. The slope of these percentile lines—reflecting the ratio $\frac{|E|}{|V|}$—varies significantly between domains. Bioinformatics graphs, for example, tend to have higher densities as indicated by a steeper slope, whereas molecular networks generally exhibit lower densities. These observations emphasize the

structural diversity present in the Oracle Dataset, making it a valuable resource for evaluating subgraph counting methods across a broad spectrum of network types.

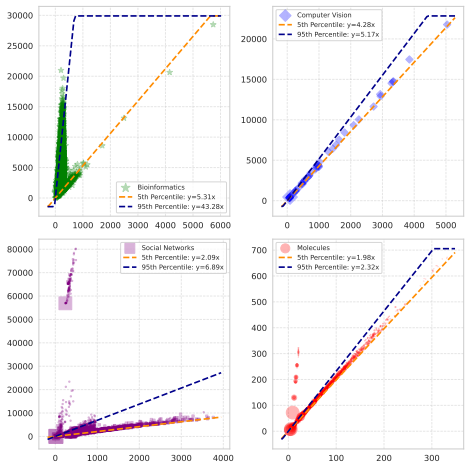



Fig. 2. Oracle Dataset's Domain and Density Distribution.

Given that the Oracle Dataset comprises so many graphs with a wide range of characteristics, it is impractical to utilize the full dataset in every research application. In many cases, researchers require a carefully selected subset that meets specific experimental criteria. For example, a study might need a sample of 10 social networks with density greater than 3 and maximum local count between 10,000 and 20,000. To meet these needs, we have developed the BEACON-Sampler, a tool that enables users to efficiently sample the Oracle Dataset based on their given constraints.

*b) BEACON-Sampler:* The BEACON-Sampler is a versatile tool for extracting graphs from a database based on specific structural and numerical criteria. Publicly available on PyPI [52] under the name *rwdq*, this tool enables researchers to fit their dataset selection through a JSON configuration file. Users can specify constraints such as minimum and maximum node counts, average degree thresholds, and other key graph properties. This configuration-driven approach permits precise filtering of graphs to match particular experimental needs. For instance, researchers can define numerical thresholds that ensure the extraction of graphs with targeted features, which is especially important in tasks like subgraph counting where algorithm performance is sensitive to underlying graph characteristics. There are 120+ pre-computed features or ground truth counts for users to set constrains on in the dataset. Furthermore, the BEACON-Sampler is designed to be compatible not only with our Oracle Dataset but also with any dataset that conforms to our established format. This flexibility supports a wide range of applications, from systematic stress testing to targeted experimentation on specific graph structures, thereby facilitating optimized and reproducible research workflows.

*2) Environment Module:* This module provides a controlled and reproducible environment for method development and evaluation. It leverages Docker containers to encapsulate the entire software stack, including dependencies, libraries, and code. By using containers, we ensure that methods can be tested and compared in identical environments (e.g. CPU set), eliminating inconsistencies due to variations in system configurations or software versions. This module also includes a Docker Hub page [52] where users can upload and share their own Docker images, ensuring that new methods adhere to the standards of reproducibility.

*3) Testing Module:* The testing module is designed to evaluate the performance of subgraph counting methods across several dimensions, including *robustness*, *scalability & efficiency*, *accuracy*, and *generalizability*. It uses a set of predefined benchmark datasets, which are representative of various scenarios, to assess how well methods perform under different conditions.

### B. Usage Scenarios

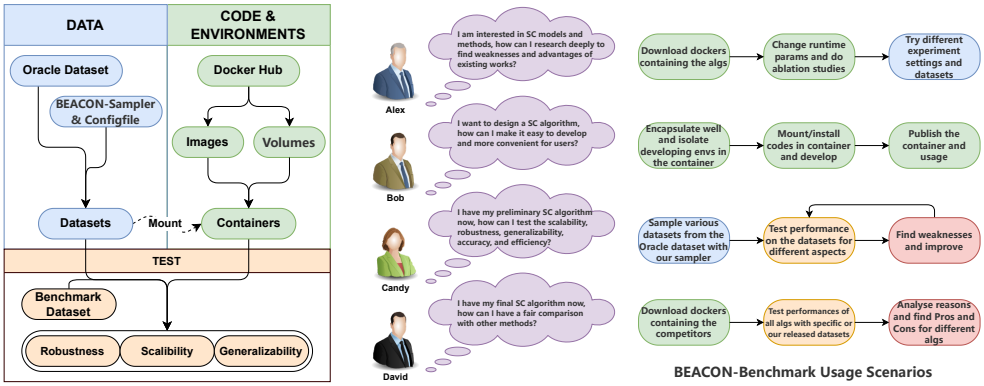
Our benchmark is designed to support a wide range of usage scenarios, allowing researchers and developers to utilize the framework for various purposes depending on their goals. Below, we outline four key usage scenarios, each addressing a different stage in the development lifecycle of a particular method for subgraph counting.

*a) Dataset Usage Only:* In this scenario, users are primarily interested in utilizing the existing ground truth datasets provided by the benchmark for their own downstream applications, without necessarily engaging in method development. For example, a researcher might use the benchmark datasets to validate a hypothesis or compare two existing solutions. The datasets come pre-labeled with ground truth subgraph counts, allowing users to focus on their specific application without the need for further data processing.

*b) Explore the Literature:* This scenario targets users who are new to the field of subgraph counting and want to explore the existing literature and techniques. Users can download the datasets and images, modify parameters, and experiment with different settings to gain a deeper understanding of how they work. Additionally, users can access the results of our benchmark tests and compare them to existing methods.

*c) Method Development:* In this scenario, users are interested in developing new methods. They can use the benchmark to perform a literature review, explore existing results, and use our datasets for preliminary testing. The BEACON-Sampler allows users to generate datasets customized for their method's needs, providing a flexible platform for experimentation. Once the method is developed, users can use the framework's leaderboards to compare their method's performance against state-of-the-art methods.

*d) Method Benchmarking:* This scenario is designed for users who already have a fully developed and nearly publishable method. They can use the benchmark to thoroughly evaluate their method's strengths and weaknesses in a fair and controlled environment. By comparing their method against others in the benchmark, users can identify areas for improvement and gain recognition on the benchmark's leaderboard. This scenario is particularly suited for researchers looking to publish their work in competitive venues.




(a) Framework (b) Scenarios

Fig. 3. (a) The framework of the proposed benchmark, and (b) different usage scenarios.

Fig. 3(b) illustrates these scenarios, showing how different users (e.g., Alex, Bob, Candy, and David) interact with the benchmark framework to achieve their respective goals. Each user represents a different stage in the method development lifecycle, from initial exploration to final benchmarking.

Beyond usage scenarios, the benchmark is designed to encourage sustained contributions from the research community, thereby ensuring its continuous evolution. Contributions may take several forms: the incorporation of real-world or synthetic datasets with validated ground-truth subgraph counts to broaden representativeness; the refinement of existing datasets through extended or more complex subgraph enumeration to enhance benchmarking depth; the improvement of the open-source pipeline via performance optimizations, feature development, or maintenance; and the promotion of reproducibility by publishing Dockerized implementations. Collectively, these avenues of contribution strengthen the benchmark's diversity, accuracy, technical robustness, and reproducibility.

### C. Modular Design Benefits

The modular design of our benchmark offers several key advantages. First, it provides flexibility by allowing users to interact with the framework at various stages of the method development lifecycle, from dataset curation and preparation to full-scale benchmarking. Second, reproducibility is ensured through the use of Docker containers, which guarantee that all methods are tested in identical environments, thereby eliminating variability due to system configuration differences. Third, our framework scales gracefully with the complexity of the task: the BEACON-Sampler supports the generation of datasets in a wide range of sizes and complexities, making it suitable for evaluating methods designed for large-scale graphs. Additionally, the open-source and community-driven nature of the benchmark encourages ongoing contributions and improvements. Finally, our design provides a dedicated avenue for application case studies, addressing a critical need in the field where subgraph analysis can considerably benefit from real-world validation and use-case enhancement.

## V. Experimental Evaluation

In this section, we showcase how our benchmark framework and curated datasets can be used to extract meaningful insights from the literature. Using our comprehensive platform and datasets, we easily compare diverse subgraph counting methodologies and reveal trends and nuances that might otherwise remain hidden. This approach not only promotes transparency and reproducibility but also underscores the essential role of a unified benchmark in advancing research and collaboration within the subgraph counting community. The experiments were conducted on Ubuntu Server 22.04 equipped with an Intel Core i9-14900KF processor, 256 GB of RAM, and dual NVIDIA RTX 4070 SUPER GPUs.

### A. Dataset

Building on the foundations provided by the Oracle Dataset and the flexibility offered by the BEACON-Sampler, we now present the Benchmark Dataset.

*a) Benchmark Dataset:* Using the BEACON-Sampler, we have curated a *Benchmark Dataset* to evaluate subgraph counting algorithms from multiple angles, including scalability, accuracy, and sensitivity. The dataset is organized into sets with distinct constraints, as detailed in Table I, which also reports total node and edge statistics in the datasets.

Sets 1 through 4 are dedicated to assess performance on smaller graphs with varying densities. These sets are split into training, validation, and test subsets in a 4:1:1 ratio, preserving a similar distribution across each. In Sets 1, 3, and 4, a maximum degree of 100 is enforced to obtain graphs with progressively increasing density. Dense graphs tend to exhibit features like cliques—challenging patterns that often remain absent in sparser datasets such as MUTAG [53]. In contrast, Set 2 impose no maximum degree limit compared with Set 1, better reflecting real-world networks where highly connected central nodes are common. This division tests the performance of different methods on dataset containing many small graphs. For AL algorithms, we parallel the computation by assigning tasks to different CPU cores, while for ML

methods, they naturally compute a batch of small graphs together to sufficiently use GPU.

Sets 5 through 8 comprise larger graphs, divided into training, validation, and test subsets in a 4:1:4 ratio. The graphs in test subsets are then combined into a single large graph for scalability experiments. These sets are designed to monitor how inference time and accuracy evolve with increasing graph size—helping to identify whether an algorithm scales efficiently or encounters resource constraints with larger graphs. For these datasets, AL algorithms must run on a single CPU core unless they support parallelization (e.g. MOTIVO allows multith-readed sampling). Since the test graph is one large graph rather than many smaller ones, ML methods must load the entire graph into memory, which places additional pressure on memory usage.

Overall, this dual approach, which combines many small graphs with one larger graph ensures that our Benchmark Dataset provides a comprehensive framework to test the computational challenges inherent in subgraph counting.

TABLE I
BENCHMARK DATASET.

| | | **Set 1** | **Set 2** | **Set 3** | **Set 4** |
|---|---|---|---|---|---|
| Stat. | #nodes | 144,669 | 265,115 | 250951 | 191,132 |
| | #edges | 159,974 | 300,322 | 665,732 | 1,475,888 |
| Cons. | #nodes/graph | 0-500 | 0-500 | 0-500 | 0-500 |
| | avg. deg. | 0-1.5 | 0-1.5 | 1.5-5 | 5-10 |
| | max. deg. | 100 | - | 100 | 100 |
| | #graphs | 1200 | 1200 | 1200 | 1200 |
| | | **Set 5** | **Set 6** | **Set 7** | **Set 8** |
| Stat. | #nodes | 87,476 | 213,726 | 435,134 | 890,482 |
| | #edges | 250,929 | 684,537 | 1,425,950 | 2,927,573 |
| Cons. | #nodes/graph | 100-5000 | 100-5000 | 100-5000 | 100-5000 |
| | #graphs | 200 | 500 | 1000 | 2000 |

### B. Setup

In this section, we detail the experimental setup used to evaluate the performance of different methods using our benchmark. A structured and transparent setup is essential to ensure reproducibility and to provide a clear understanding of the conditions under which our results were obtained. This includes the patterns used for testing, the competitors, and the training procedures we employed. Each component of the setup has been carefully chosen to reflect real-world scenarios and to provide comprehensive insights into the method's capabilities under different conditions. First, we describe the graph patterns used in our experiments and the rationale behind selecting them, followed by an overview of the competing methods we tested against. Finally, we present the details of our training process, including the parameters used and the learning strategies explored.

*a) Tested Patterns:* To evaluate the performance of each method, we use a diverse set of patterns ranging in size from 3 to 5 nodes. These sizes were chosen because they provide the most valuable insights in practical applications [8] and are supported by all competing algorithms. In addition, we assess performance on larger patterns with up to 8 nodes in Section V-C. The specific patterns used in our experiments are shown in Fig. 4. For clarity, we will refer to these patterns as "target" moving forward. For instance, target 1 refers to the triangle.

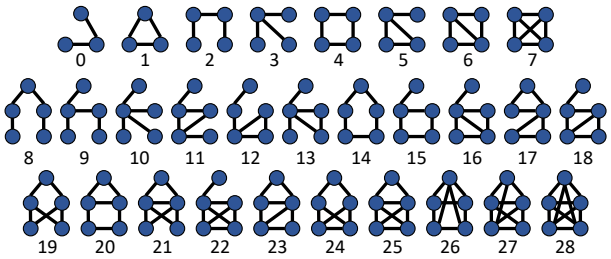



Fig. 4. Tested patterns.

*b) Competitors:* As shown in Table II, we evaluated a broad range of subgraph counting methods using our benchmark, classifying them into three categories: exact, approximate, and ML-based. Exact algorithms, such as ESCAPE [24], can compute accurate subgraph counts but are constrained by computational limitations, particularly for larger subgraph sizes (denoted by the parameter $k$). For EVOKE and SCOPE [40], which are used for orbit counting, we aggregate orbit counts of all the different orbits within a specific pattern to obtain the local subgraph counts. For example, target 10 has two different orbits, center and periphery. A node belongs to the instance of the target regardless of the orbit it occupies. To compute the global counts, we aggregate the local counts and normalize by the number of nodes in the target. This normalization is required because each instance of the target is evaluated $|V|$ times, where $|V|$ denotes the cardinality of its node set. Furthermore, as they compute non-induced counts, we convert non-induced counts to induced counts by matrix multiplication. For detailed matrix information, readers may consult [24], [25]. The above pipeline is also available in our code repository, where we fixed matrix errors present in [25], which would otherwise result in incorrect counts. Fig. 5 illustrates the conversion between different counting results. The detailed description is omitted due to space constraints. One important point to note is that converting between induced and non-induced counts requires more than just the counts of the target itself. It may also depend on the counts of other targets. For example, each triangle (target 1) contains three instances of target 0. Thus, when converting the induced count of target 0 to the non-induced count, the count of target 1 is also needed. Since the above algorithms support network-centric counting, conversions between all targets of the same size can be executed. Nonetheless, counting larger induced patterns remains challenging for existing algorithms, since the number of orbits (or patterns) grows exponentially with $k$. For $k = 5, 6, 7$, the numbers of orbits are $58$, $407$, and $4306$, respectively. Thus, obtaining induced counts by computing non-induced counts of all different orbits is impractical, especially for large size $k$. Approximate algorithms, like MOTIVO [26], prioritize speed over accuracy, making

them suitable for larger graphs. Finally, ML-based algorithms, including GNN-based methods like PPGN [47] and ID-GNN [37], harness ML to offer scalable and adaptable solutions.

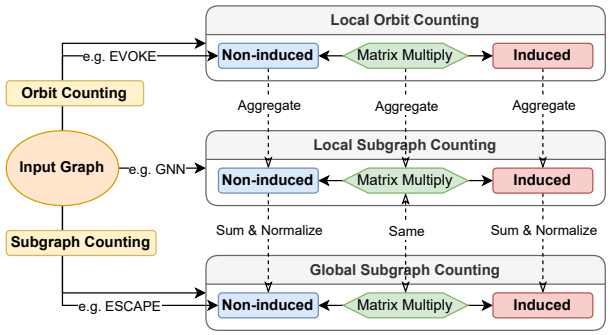



Fig. 5. Counting conversion.

Table II categorizes these methods based on year, k-restriction ("-" indicates no theoretical restriction), type, induction type (I.T.: non-induced (N), induced (I), or both (B)), counting process (C.P.: local (L) or global (G) subgraph counting), centricity type (C.T.: subgraph-centric (S) or network-centric (N)), and code repositories.

TABLE II
OVERVIEW OF METHODS.

| Algorithm | Year | k | Type | I.T. | C.P. | C.T. | Code |
|---|---|---|---|---|---|---|---|
| ESCAPE [24] | 2017 | 5 | Exact | N | G | N | [54] |
| EVOKE [25] | 2020 | 5 | Exact | N | L | N | [55] |
| SCOPE [40] | 2024 | - | Exact | N | L | S&N | [56] |
| MOTIVO [26] | 2019 | - | Approx. | I | G | N | [57] |
| GIN [58] | 2019 | - | ML-based | B | L | S | [59] |
| PPGN [47] | 2021 | - | ML-based | B | L | S | [60] |
| GNN-AK$^+$ [46] | 2022 | - | ML-based | B | L | S | [61] |
| I$^2$-GNN [48] | 2022 | - | ML-based | B | L | S | [62] |
| ID-GNN [37] | 2022 | - | ML-based | B | L | S | [63] |
| ESC-GNN [38] | 2024 | - | ML-based | B | L | S | [64] |
| DeSCo [49] | 2024 | - | ML-based | B | L | S | [65] |

*c) Training Process:* The selection of hyperparameters is critical for achieving optimal performance and ensuring a fair comparison across all algorithms. We chose the hyperparameters based on both empirical evaluations and guidelines from the literature (suggestions from the authors), aiming to balance accuracy, training time, and generalization. For each method, we conducted a hyperparameter search within reasonable ranges, taking into account factors such as the number of layers, learning rate, and batch size. Further details about the specific parameters used for each algorithm are available on our code repository.

In a typical subgraph counting task on a static graph, the counting operation only needs to be performed once. Training a model on the same graph beforehand is impractical for two reasons. First, the training process is time-consuming. Second, effective training requires access to at least a portion of the ground truth, and if that is available, then the use of an ML model for prediction becomes unnecessary. Instead, an ideal ML approach is one that is trained in advance using established datasets, and can then accurately predict subgraph counts on completely unseen graphs.

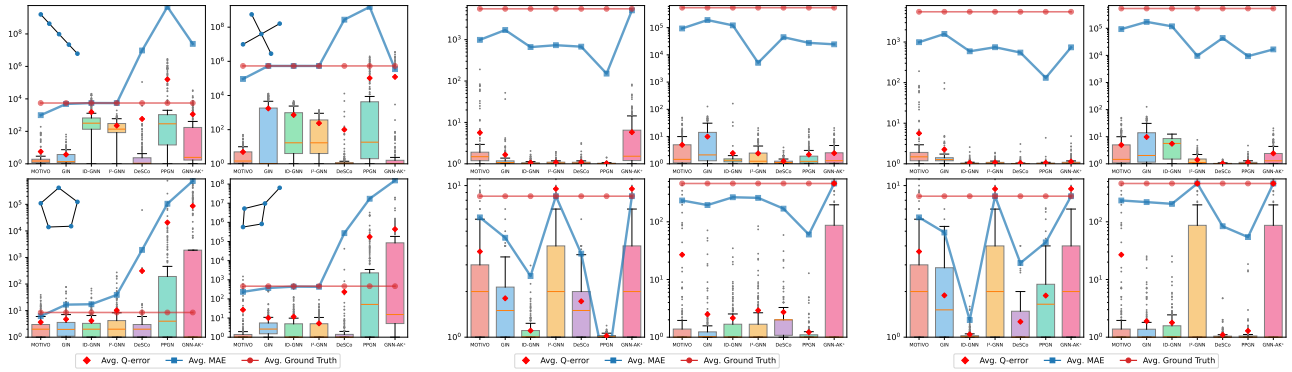
To assess this capability, we adopt a zero-shot evaluation where models that are pre-trained on separate data are directly applied to our Benchmark Dataset. We also consider scenarios where a model is given a limited look at the new data where a pre-trained model is fine-tuned for a few epochs. Finally, we evaluate each algorithm under a full training setup to observe its peak performance, even though such a scenario is less practical for static graphs. In what follows, we discuss these evaluation scenarios in detail. We use the same process in [48] to pre-train our models. Our evaluation strategy is divided into three scenarios:

1) *Zero-shot Evaluation*: In this phase, we test the pre-trained model directly on the Benchmark Dataset without any additional fine-tuning. This approach evaluates the model's ability to generalize to the completely unseen graphs and confirms that it has learned robust subgraph counting patterns rather than simply memorizing its training data.
2) *Fine-tuning*: Here, we fine-tune a pre-trained model using a portion of the new dataset with 10% of the full training epochs. This limited fine-tuning examines how effectively the model leverages its prior knowledge to quickly adapt to new data and improve its accuracy.
3) *Full Retraining*: Lastly, models are fully trained from scratch to determine their best achievable performance. Although this setup is less practical for static graphs where the counting task is performed only once, it provides a valuable baseline for comparing the peak performance of different models.

## C. Results

In this section, we present the experimental results of our benchmark. We evaluate the performance of our proposed method in terms of efficiency, accuracy, and robustness. We begin by assessing the runtime performance of each method to measure their computational efficiency. Next, we evaluate how accurately the models estimate subgraph counts across various graph types. Finally, we examine the models' robustness by testing them under different graph conditions, which helps us understand their ability to generalize to unseen patterns and datasets. Each part of the evaluation provides key insights into the strengths and limitations of the methods.

*1) Efficiency:* We evaluate the efficiency of our algorithms across various datasets and pattern complexities. The overall runtime is broken down into three phases: *training*, *pre-processing*, and *inference*. Pre-processing and inference times are highlighted in Fig. 7, while Table III shows a detailed comparison of the training times for models that require it. This separation is important because, in practical scenarios, a well-trained model should be able to generalize to new graphs without any additional fine-tuning, which means only pre-processing and inference would be needed. If additional training is required, it indicates that ground truth data must be




(a) Zero-Shot (b) Fine-tuning (c) Full Retraining

Fig. 6. Accuracy under different training scenarios on Set 1.

accessible, which would defeat the purpose of using an ML model for prediction in the first place.

TABLE III
TRAINING TIME COMPARISON (MINUTES).

| Algorithms | Set 1 | Set 2 | Set 3 | Set 4 | Set 5 | Set 6 | Set 7 | Set 8 |
|---|---|---|---|---|---|---|---|---|
| **DeSCo-ST** | 269 | 500 | 616 | 1705 | 734 | 2069 | 3578 | 12167 |
| **DeSCo** | 22.7 | 37.8 | 47.6 | 102 | 44.2 | 110 | 211 | 639 |
| **GIN** | 2.37 | 4.52 | 2.97 | 3.24 | 1.06 | 1.86 | 3.59 | 9.66 |
| **GNN-AK$^+$** | 81.4 | 255 | 145 | 336 | N/A | N/A | N/A | N/A |
| **ID-GNN** | 40.7 | 145 | 86.8 | 238 | N/A | N/A | N/A | N/A |
| **ESC-GNN** | 33.5 | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| **I$^2$-GNN** | 118 | N/A | N/A | N/A | N/A | N/A | N/A | N/A |
| **PPGN** | 636 | 1410 | 1197 | 446 | N/A | N/A | N/A | N/A |

*a) Training Time:* Table III presents a comparison of training times for the different methods across datasets of various sizes. Here, "N/A" indicates that a method failed during training, primarily due to excessive GPU memory usage. It is important to emphasize that the reported training times are averaged over all 29 targets, meaning that the values indicate the average time required to train the model for a single target. DeSCo was originally designed to train on all targets concurrently. To facilitate a fair comparison, we modified DeSCo to focus on a specific target by training sequentially on each target and summing the training times. The corresponding training time for a single target under this configuration is noted as DeSCo-ST (Separate Target).

The results reveal several key trends, one of which is that GIN exhibits the lowest training time among all methods. This efficiency comes from its simple architecture, making it the most scalable model as graph sizes increase. Such scalability offers promise for quick training and deployment in real application although its discrimination power is limited to 1-WL [58]. PPGN always exhibits the highest training times across datasets. This stems from its 2-order graph network architecture, which stores feature vectors for every node pair. As a result, it requires quadratic memory in the number of nodes and incurs an $O(n^3)$ time complexity due to matrix multiplications. This is substantially more expensive than the linear or near-linear complexity of message-passing GNNs on sparse graphs, and the gap becomes even more pronounced for single large graphs with many nodes, such as in Set 5–8. On these datasets, PPGN runs out of memory (OOM) easily. Another interesting observation is that different methods respond inconsistently to graph density, number of nodes, and number of edges. For instance, PPGN shows little sensitivity to density or edge count but are strongly affected by the number of nodes, consistent with the above analysis. However, the training time of GNN-AK$^+$ and ID-GNN generally increases from Set 1 to Set 4, except for Set 2. This exception occurs because Set 2 imposes no maximum degree limit, resulting in many large repeated star-shaped ego-networks. At the same time, we see that DeSCo does not exhibit this trend, as its ID-constrained ego-network splitting method effectively reduces redundancy, which also makes DeSCo trainable on large single graphs (Set 5-8) while other methods (except GIN) fail.

TABLE IV
RUNTIME FOR SET 5-8.

| Dataset | ESCAPE | EVOKE | SCOPE | MOTIVO | DeSCo | GIN |
|---|---|---|---|---|---|---|
| **Set 5** | 0.83 | 1.57 | 1.08 | 2.67 | 115.16 | 0.18 |
| **Set 6** | 85.0 | 76.3 | 20.5 | 4.73 | 302.43 | 0.45 |
| **Set 7** | 327.24 | 295.52 | 72.06 | 21.29 | 914.69 | 1.78 |
| **Set 8** | 495.31 | 449.00 | 114.54 | 38.59 | 1606.99 | 4.09 |

*b) Pre-processing and Inference Time:* The pre-processing and inference times of the evaluated algorithms on different datasets are illustrated in Fig. 7 and Table IV. In practical applications, where models are already well-trained and do not require additional fine-tuning, only the steps of preparing the data (pre-processing) and making predictions (inference) are needed. Subgraph GNNs (GNN-AK$^+$, ID-GNN, DeSCo, I$^2$-GNN) require pre-processing by extracting ego-networks around each node, often with additional feature computation. For instance, DeSCo builds ID-constrained ego-networks and augments by indicating triangle instances, while ESC-GNN enriches node features using distance-based information. We can see that the pre-processing is very time-consuming and accounts for large proportion in running time,

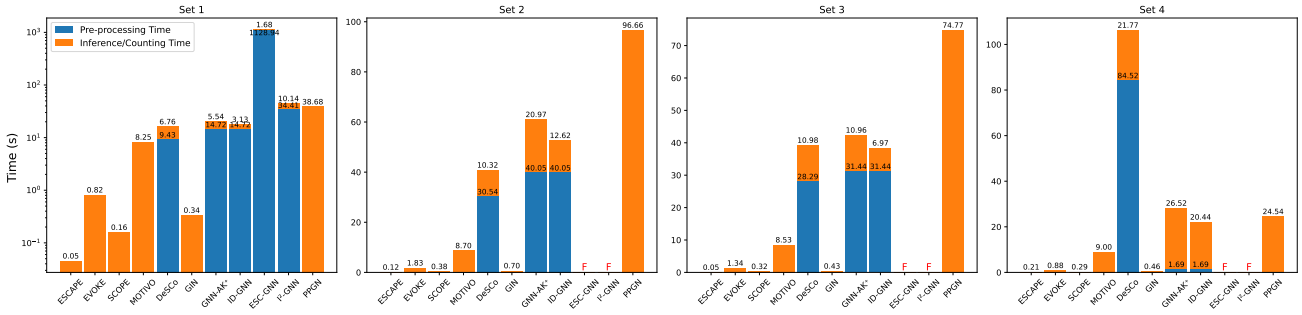



Fig. 7. Runtime time (red "F" indicates failure) for Set 1-4.

even longer than the time exact methods consume for full run. Thus, current architectures still have room for improvement. From the comparison we can find that AL-based exact and approximate methods are very efficient compared with subgraph GNN methods, thus exact methods achieve both efficiency and accuracy advancement for small patterns while approximate method MOTIVO is even faster. Additionally, we can find that among AL algorithms, EVOKE and SCOPE demonstrate better efficiency compared with ESCAPE for single large graph when the graph is large enough (Set 6-8). This indicates that using orbit counting may also stand as a powerful subgraph counting solution for large graphs. Regarding GIN, it demonstrates great scalability and efficiency on Set 6-8 while most of ML-based methods fail. Note that the reported running time accounts for all targets. If only a specific target is of interest, the performance gap between GIN and network-centric algorithms becomes even larger. Its sublinear growth in running time suggests that MPNNs could be developed further to handle larger graphs efficiently, especially if the current challenges with accuracy (related to its limited discriminative power with the 1-WL framework) can be overcome.

*2) Accuracy and Precision:* We evaluate the accuracy of existing algorithms across three different scenarios, each designed to highlight a specific aspect of the method's performance. Fig. 6 presents boxplots comparing the performance of the methods under these scenarios where we select four representative targets with highly distinct structures for illustration. The evaluation metrics include the average Q-error, average MAE, and average ground truth. It is important to note that MOTIVO, a non-learning approximate counting algorithm, remains unaffected by these scenarios. Rather, it serves as a baseline to indicate the accuracy scale and facilitate comparisons with the learning-based methods.

*a) Zero-shot Test:* The zero-shot test (Fig. 6 (a)) shows that the learning ability of the existing algorithms varies significantly among different targets. For example, the Q-error of ID-GNN exceeds $10^3$ on target 8 (5-node path), but remains below 3 on target 14 (5-node circle). This is because, in ID-GNN, the identity of the ego-network center can be readily identified, which allows the message-passing process to distinguish the center from its neighbors and thereby facilitates the detection and learning of circular structures. By comparing the MAE and Q-error results, we can see that the methods are evaluated differently on the two metrics. GIN, ID-GNN, and $I^2$-GNN have similar MAE on target 10 in zero-shot test, while their Q-errors are distinct, which stems from the characteristics of MAE and Q-error that the same absolute difference may lead to distinct relative bias. As indicated by the boxplots and medians, DeSCo predicts values accurately and close to 1 in most cases; however, the presence of many outliers leads to a larger average Q-error. This characteristic arises from its ID-constrained ego-networks. The model tends to generate many small and simple ego-networks that are frequently observed during training. However, when faced with an unseen and more complex ego-network, it struggles to generalize effectively. In general, the zero-shot results indicates that the pre-trained models are not performing well (compared with MOTIVO) when the distribution changes.

*b) Fine-tuning and Retraining Test:* After fine-tuning, the predictive accuracy improves substantially. The average MAE remains consistently lower than the average ground truth, and most methods achieve an average Q-error of less than $10^1$. A majority of the models outperform MOTIVO in terms of MAE or Q-error, suggesting their ability to adapt to new datasets. The performance after retraining (Fig. 6 (c)) is slightly better than fine-tuning (Fig. 6 (b)). For instance, ID-GNN nearly gets the exact counts after retraining. However, we can also notice a few cases does not show better results of retrain than fine-tuning. The performance of PPGN on circle after retraining is worse than fine-tuning. This can happen when: (1) learning a new distribution from scratch is more difficult and converges more slowly; (2) hyper-parameters optimized for the previous dataset/distribution do not guarantee good performance on the new distribution; (3) randomness introduces uncertainty during the training phase. The above analysis also reveals the common drawbacks for ML-based subgraph counting methods, which we will have more detailed conclusion in Section V-D.

*3) Robustness and Generalization:* In this section, we evaluate the robustness and generalization capabilities of the methods under various conditions.

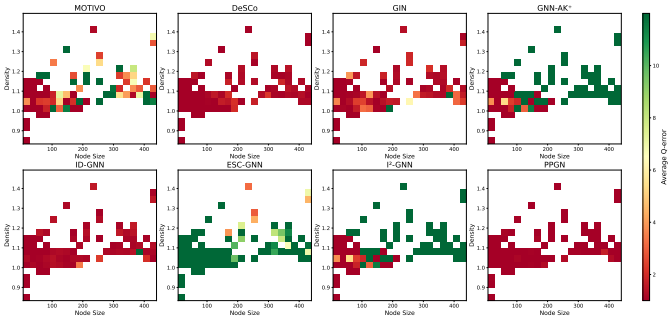
*a) Graph Structure Impact on Accuracy:* We start by examining how different graph structural properties, including degree and density, influence the model's accuracy. Table V

TABLE V
FINE-TUNING Q-ERROR ON TARGET 13.

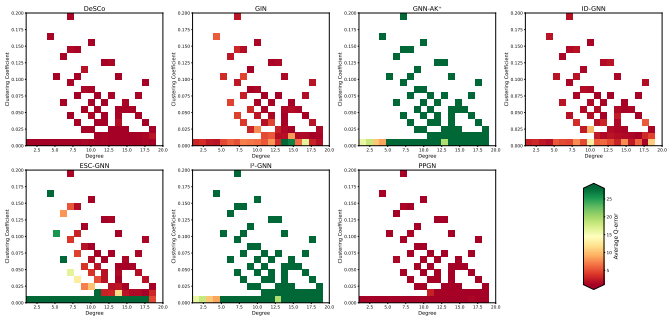
| Algorithms | Set 1 | Set 2 | Set 3 | Set 4 |
|---|---|---|---|---|
| **MOTIVO** | 108.88 | 469.86 | 2.43 | 1.13 |
| **DeSCo** | 1.93 | 1.39 | 1.13 | 1.21 |
| **GIN** | 8.33 | 24.63 | 7.69 | 2.23 |
| **GNN-AK$^+$** | 4307.59 | 7612.56 | 173.02 | 1438.93 |
| **ID-GNN** | 4.45 | 26.48 | 1.12 | 1.76 |
| **ESC-GNN** | 29220.07 | N/A | N/A | N/A |
| **I$^2$-GNN** | 5.01 | N/A | N/A | N/A |
| **PPGN** | 1.02 | 4.01 | 1.39 | 1.64 |

reports the fine-tuned Q-error accuracy for target 13 across different datasets. Notably, the Q-error on Set 2, where the dataset imposes no degree constraint, is substantially higher than on the other datasets. This result highlights the difficulty models face when predicting on graphs that contain a few very high-degree nodes. The node degree distribution of the dataset graphs (available in our code repository) follows a clear power-law pattern, which explains this challenge. In real-world networks (e.g. social networks), node and graph properties span a wide range and are inherently non-uniformly distributed. For instance, training data typically includes many low-degree nodes, while high-degree nodes become increasingly scarce as degree increases. As a result, the model is exposed primarily to low-degree nodes during training and lacks sufficient examples of high-degree nodes. Yet, as ego-networks grow more complex, the number of possible structural variations expands super-exponentially with node degree. This leads to a mismatch: abundant training data exists for simple, low-degree structures, but relatively few examples are available for high-degree cases where the structural space is much larger. Consequently, models struggle to generalize to high-degree nodes. Importantly, artificially enforcing a uniform degree distribution is not a viable solution, as it would fail to capture the true characteristics of real-world networks.

An exception to the aforementioned trend is DeSCo, whose performance is not significantly affected by high-degree nodes. This robustness arises from its ego-network splitting method, which simplifies the structure of the ego-network and can even reproduce structures previously seen in the training phase, particularly when high-degree nodes have large IDs. Combined with analysis of DeSCo in previous sections, our computation and provision of the ground truth for ID-constrained ego-networks are further justified. Unlike the other models, MOTIVO exhibits poor Q-error primarily due to sampling failures or undersampling. Target 13 consists of a triangle and two edges. For high-degree nodes with low clustering coefficients (i.e. neighbors are less likely to be connected), MOTIVO often fails to capture the triangle component while oversampling star-shaped structures. Because there are numerous unconnected neighbor pairs, the number of target 13 instances is large, which contributes to the inaccuracy observed in Set 1 and, more severely, in Set 2. As graph density increases, however, triangle sampling becomes more reliable, leading to improved Q-error performance across Sets 1, 3, and 4.




(a) Graph Accuracy




(b) Node Accuracy

Fig. 8. Effect of graph size, density, node degree and clustering coefficient on accuracy on target 11.

*b) Graph- and Node- Level Accuracy:* Fig. 8 illustrates the impact of graph-level (Fig. 8(a)) and node-level (Fig. 8(b)) characteristics on counting target. In Fig. 8(a), the x-axis indicates the graph size (in terms of $|V|$) and the y-axis reports the graph density ($\frac{|E|}{|V|}$). The color of each point corresponds to the Q-error with lower values being better. This plot demonstrates how each method reacts to global graph features. Similarly, Fig. 8(b) focuses on local graph properties. The x-axis shows the node degree, and the y-axis corresponds to the node's clustering coefficient. The results indicate that larger graphs and lower density pose greater prediction challenges for some models (e.g. GNN-AK$^+$, I$^2$-GNN). Regarding local structural properties, graphs with lower clustering coefficient and higher node degrees also tend to be more difficult. Although target 11 differs slightly from target 13, their structural similarities lead to comparable effects under the aforementioned characteristics. Since the detailed reasoning provided for target 11 applies here, it is not repeated.

*c) Finetuning and Adaptability:* We further assess the ability of different models to adapt and finetune to specific tasks, focusing on their performance on a particular target. Fig. 9 presents the finetuning results for different models on target 11. The scatter plot shows that PPGN demonstrates superior finetuning capabilities, closely aligning their predictions with the ground truth. These results emphasize the model's generalization ability and its capacity to adjust to new datasets.

*d) Accuracy vs. Pattern Size:* We compare the fine-tuning performance of different models on varying subgraph patterns (Table VI) on Set 1, focusing on star-shaped patterns. $n$-star means the pattern that has a center node connected with $n-1$

TABLE VI
STAR PATTERN ACCURACY WITH MAE AND Q-ERROR FOR STAR-SHAPED PATTERNS.

| Alg | 3-star | | 4-star | | 5-star | | 6-star | | 7-star | | 8-star | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MAE | Q-error | MAE | Q-error | MAE | Q-error | MAE | Q-error | MAE | Q-error | MAE | Q-error |
| **MOTIVO** | $1.1e+02$ | 1.23 | $4.0e+03$ | 1.72 | $1.1e+05$ | 4.97 | $4.5e+07$ | 2.80 | $7.6e+08$ | 6.15 | $1.1e+10$ | 87.0 |
| **DeSCo** | $1.7e+02$ | 1.07 | $2.9e+03$ | 1.07 | $4.4e+04$ | 1.25 | $1.9e+07$ | 1.35 | $5.0e+08$ | 6.33 | $4.6e+09$ | 10.4 |
| **GIN** | $2.8e+02$ | 1.10 | $4.3e+03$ | 1.13 | $3.8e+05$ | 2.66 | $8.9e+06$ | 38.0 | $1.3e+08$ | $1.8e+03$ | $1.5e+09$ | $2.5e+04$ |
| **GNN-AK$^+$** | $4.7e+02$ | 1.21 | $2.0e+02$ | 1.01 | $1.8e+04$ | 1.48 | $5.8e+05$ | 1.67 | $1.3e+07$ | 1.12 | $1.4e+08$ | 1.54 |
| **ID-GNN** | $3.4e+01$ | 1.01 | $2.5e+03$ | 1.10 | $7.0e+04$ | 1.42 | $3.1e+06$ | 1.92 | $5.3e+07$ | 2.56 | $7.2e+08$ | 9.00 |
| **ESC-GNN** | $2.2e+03$ | 10.0 | $2.5e+04$ | 44.4 | $3.1e+05$ | $8.0e+02$ | $8.3e+06$ | $7.8e+03$ | $1.2e+08$ | $7.8e+03$ | $1.5e+09$ | $3.1e+04$ |
| **I$^2$-GNN** | $5.5e-01$ | 1.00 | $1.1e+01$ | 1.01 | $1.8e+03$ | 2.11 | $7.9e+04$ | 1.01 | $2.9e+06$ | 1.54 | $3.0e+07$ | 1.36 |
| **PPGN** | $5.2e+02$ | 1.65 | $5.1e+03$ | 1.20 | $6.5e+03$ | 1.22 | $4.9e+05$ | 1.68 | $2.9e+06$ | 1.22 | $5.9e+08$ | 1.62 |

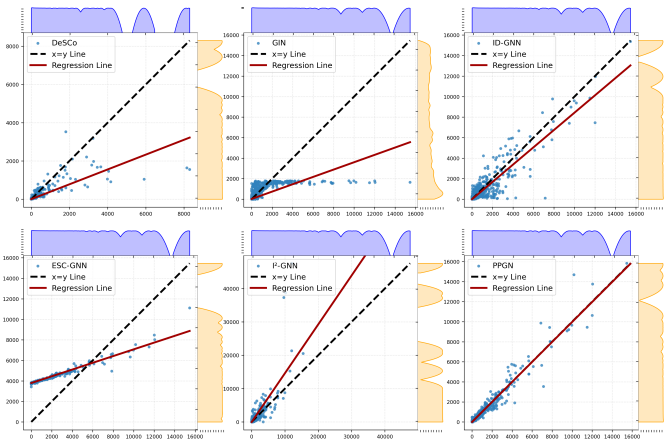



Fig. 9. Prediction and ground truth on Set 1, target 11.

periphery nodes (e.g. 3-star is target 0, 5-star is target 10). We observe that as the pattern size increases, the MAE also rises. This trend results from the sharp expansion of the ground-truth counting range, which poses greater challenges for the models, as discussed in Section V-C3a. However, models such as I$^2$-GNN and GNN-AK$^+$ demonstrate good scalability in terms of Q-error, suggesting that ML-based approaches hold promise for large-pattern predictions, where traditional algorithms often encounter limitations.

### D. Discussion and Summarization

*a) Traditional Algorithms:* Traditional methods provide highly accurate results for small patterns, with exact algorithms offering efficient query performance. Approximation algorithms can also strike a favorable balance between accuracy and runtime.

However, exact methods often struggle to scale to large patterns. Induced and non-induced counts are not interchangeable without counts of other specific patterns, and the conversion between induced and non-induced counts becomes more infeasible for larger patterns as discussed in Section V-B. Sampling-based methods may generate inaccurate approximation due to sample failure.

*b) ML-based Methods:* ML-based approaches for subgraph counting naturally support arbitrary patterns, including both induced and non-induced counting. Their runtime is relatively insensitive to pattern structure, primarily influenced by the pattern's diameter. Moreover, the expressive power of some models provides theoretical guarantees for subgraph counting. In addition to the global count, the local count they produce provides supplementary information.

Although these methods offer flexibility, they face several challenges. They generally require large amounts of training data, which is difficult to obtain due to the complexity of computing ground truth, particularly for models needing specific counts. This issue is more severe for larger patterns. Training is often time-consuming and affected by the quality and balance of the training data, as discussed in detail in Section V-D. The super-exponential growth in the number of possible local graph structures with increasing hop number and node degree poses additional challenges for developing robust pre-trained models. Additionally, hyperparameter selection can be sensitive to the datasets and prediction tasks. During inference, high-order GNNs may consume excessive memory due to the polynomial number of node tuples and suffer from long runtime. Subgraph GNNs require pre-processing, possibly including pre-computation of extra information, which extends inference time. Theoretically, their expressive power remains limited, preventing them from distinguishing certain structures.

## VI. CONCLUSIONS

In this paper, we introduced BEACON, a scalable and standardized benchmark for evaluating both traditional and machine learning-based subgraph counting algorithms. BEACON addresses key challenges of reproducibility, scalability, and generalizability. Through extensive experiments, we demonstrated its effectiveness in revealing the strengths and limitations of existing approaches, underscoring the importance of a universal benchmarking framework. Future work will explore new approaches to advance subgraph counting by integrating classical methods with learning-based techniques. One promising direction is to combine pattern decomposition, which is commonly employed in traditional algorithms, with machine learning models such as GNNs. In this approach, local counts of small patterns are used as features to guide the counting of larger patterns. By decomposing larger structures, estimating their counts, and then recombining the results, we aim to enhance both efficiency and discriminative power.

## AI-Generated Content Acknowledgement

We utilized generative large language models (the ChatGPT series, Grok 3, and Gemini 2.5 Pro) to refine the manuscript's content, generate mathematical equations based on our specifications, and assist with structural organization. All LLM-generated material was thoroughly reviewed and scrutinized by the authors.

## References


[1] L. R. Foulds, *Graph theory applications*. Springer Science & Business Media, 1995.

[2] D. M. Kane, K. Mehlhorn, T. Sauerwald, and H. Sun, "Counting arbitrary subgraphs in data streams," in *Automata, Languages, and Programming - 39th International Colloquium, ICALP, Warwick, UK, July 9-13, Proceedings, Part II*, ser. Lecture Notes in Computer Science, vol. 7392. Springer, 2012, pp. 598–609.

[3] V. Vassilevska Williams and R. Williams, "Finding, minimizing, and counting weighted subgraphs," *SIAM J. Comput.*, vol. 42, no. 3, pp. 831–854, 2013.

[4] N. K. Ahmed, T. L. Willke, and R. A. Rossi, "Estimation of local subgraph counts," in *IEEE International Conference on Big Data (IEEE BigData), Washington DC, USA, December 5-8*. IEEE Computer Society, 2016, pp. 586–595.

[5] S. Comandur and S. Tirthapura, "Scalable subgraph counting: The methods behind the madness," in *Companion of The World Wide Web Conference, San Francisco, CA, USA, May 13-17, 2019*. ACM, 2019, pp. 1317–1318.

[6] I. Bordino, D. Donato, A. Gionis, and S. Leonardi, "Mining large networks with subgraph counting," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), December 15-19, Pisa, Italy*. IEEE Computer Society, 2008, pp. 737–742.

[7] G. M. Slota and K. Madduri, "Fast approximate subgraph counting and enumeration," in *42nd International Conference on Parallel Processing, ICPP 2013, Lyon, France, October 1-4, 2013*. IEEE Computer Society, 2013, pp. 210–219.

[8] P. Ribeiro, P. Paredes, M. E. P. Silva, D. Aparício, and F. M. A. Silva, "A survey on subgraph counting: Concepts, algorithms, and applications to network motifs and graphlets," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 28:1–28:36, 2022.

[9] J. Focke and M. Roth, "Counting small induced subgraphs with hereditary properties," *SIAM J. Comput.*, vol. 53, no. 2, pp. 189–220, 2024.

[10] A. Bonifati, M. T. Özsu, Y. Tian, H. Voigt, W. Yu, and W. Zhang, "The future of graph analytics," in *Companion of the International Conference on Management of Data, SIGMOD/PODS, Santiago AA, Chile, June 9-15*. ACM, 2024, pp. 544–545.

[11] Y. Fang, W. Luo, and C. Ma, "Densest subgraph discovery on large graphs: Applications, challenges, and techniques," *Proc. VLDB Endow.*, vol. 15, no. 12, pp. 3766–3769, 2022.

[12] P. M. P. Ribeiro, F. M. A. Silva, and L. M. B. Lopes, "Efficient parallel subgraph counting using g-tries," in *Proceedings of the IEEE International Conference on Cluster Computing, Heraklion, Crete, Greece, 20-24 September*. IEEE Computer Society, 2010, pp. 217–226.

[13] P. Floderus, M. Kowaluk, A. Lingas, and E. Lundell, "Detecting and counting small pattern graphs," *SIAM J. Discret. Math.*, vol. 29, no. 3, pp. 1322–1339, 2015.

[14] B. Wackersreuther, P. Wackersreuther, A. Oswald, C. Böhm, and K. M. Borgwardt, "Frequent subgraph discovery in dynamic networks," in *Proceedings of the Eighth Workshop on Mining and Learning with Graphs, MLG, Washington, D.C., USA, July 24-25*. ACM, 2010, pp. 155–162.

[15] M. Kuramochi and G. Karypis, "Frequent subgraph discovery," in *Proceedings of the IEEE International Conference on Data Mining, 29 November - 2 December, San Jose, California, USA*, N. Cercone, T. Y. Lin, and X. Wu, Eds. IEEE Computer Society, 2001, pp. 313–320.

[16] O. Papapetrou, E. Ioannou, and D. Skoutas, "Efficient discovery of frequent subgraph patterns in uncertain graph databases," in *EDBT, 14th International Conference on Extending Database Technology, Uppsala, Sweden, March 21-24, Proceedings*. ACM, 2011, pp. 355–366.

[17] X. Chen and J. C. Lui, "Mining graphlet counts in online social networks," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 12, no. 4, pp. 1–38, 2018.

[18] L. Wang, F. V. Lin, M. Cole, and Z. Zhang, "Learning clique subgraphs in structural brain network classification with application to crystallized cognition," *NeuroImage*, vol. 225, p. 117493, 2021.

[19] A. Mrzic, P. Meysman, W. Bittremieux, P. Moris, B. Cule, B. Goethals, and K. Laukens, "Grasping frequent subgraph mining for bioinformatics applications," *BioData Min.*, vol. 11, no. 1, pp. 20:1–20:24, 2018.

[20] M. M. Najafi, C. Ma, X. Li, R. Cheng, and L. V. S. Lakshmanan, "Moser: Scalable network motif discovery using serial test," *Proc. VLDB Endow.*, vol. 17, no. 3, p. 591–603, Nov. 2023. [Online]. Available: https://doi.org/10.14778/3632093.3632118

[21] C. Kosyfaki, N. Mamoulis, E. Pitoura, and P. Tsaparas, "Flow motifs in interaction networks," in *Advances in Database Technology - 22nd International Conference on Extending Database Technology, EDBT, Lisbon, Portugal, March 26-29*. OpenProceedings.org, 2019, pp. 241–252.

[22] J. R. Ullmann, "An algorithm for subgraph isomorphism," *J. ACM*, vol. 23, no. 1, pp. 31–42, 1976. [Online]. Available: https://doi.org/10.1145/321921.321925

[23] S. A. Cook, "The complexity of theorem-proving procedures," in *Logic, Automata, and Computational Complexity: The Works of Stephen A. Cook*, ser. ACM Books, B. M. Kapron, Ed. ACM, 2023, vol. 43, pp. 143–152. [Online]. Available: https://doi.org/10.1145/3588287.3588297

[24] A. Pinar, C. Seshadhri, and V. Vishal, "ESCAPE: efficiently counting all 5-vertex subgraphs," in *Proceedings of the 26th International Conference on World Wide Web, WWW, Perth, Australia, April 3-7*. ACM, 2017, pp. 1431–1440.

[25] N. Pashanasangi and C. Seshadhri, "Efficiently counting vertex orbits of all 5-vertex subgraphs, by EVOKE," in *WSDM: The Thirteenth ACM International Conference on Web Search and Data Mining, Houston, TX, USA, February 3-7*. ACM, 2020, pp. 447–455.

[26] M. Bressan, S. Leucci, and A. Panconesi, "Motivo: Fast motif counting via succinct color coding and adaptive sampling," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1651–1663, 2019.

[27] P. A. Papp and R. Wattenhofer, "A theoretical comparison of graph neural network extensions," in *International Conference on Machine Learning, ICML, 17-23 July, Baltimore, Maryland, USA*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 17 323–17 345.

[28] C. Qian, G. Rattan, F. Geerts, M. Niepert, and C. Morris, "Ordered subgraph aggregation networks," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems, NeurIPS, New Orleans, LA, USA, November 28 - December 9*, 2022.

[29] D. Sandfelder, P. Vijayan, and W. L. Hamilton, "Ego-gnns: Exploiting ego structures in graph neural networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, Toronto, ON, Canada, June 6-11*. IEEE, 2021, pp. 8523–8527.

[30] T. Schwabe and M. Acosta, "Cardinality estimation over knowledge graphs with embeddings and graph neural networks," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 44:1–44:26, 2024.

[31] L. Sun, Z. Huang, Z. Wang, F. Wang, H. Peng, and P. S. Yu, "Motif-aware riemannian graph neural network with generative-contrastive learning," in *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI , Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI, February 20-27, Vancouver, Canada*. AAAI Press, 2024, pp. 9044–9052.

[32] X. Zhang, Y. Shen, Y. Shao, and L. Chen, "DUCATI: A dual-cache training system for graph neural networks on giant graphs with the GPU," *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 166:1–166:24, 2023.

[33] Z. Zhang, J. Bu, M. Ester, J. Zhang, Z. Li, C. Yao, H. Dai, Z. Yu, and C. Wang, "Hierarchical multi-view graph pooling with structure learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 545–559, 2023.

[34] L. Zhao, W. Jin, L. Akoglu, and N. Shah, "From stars to subgraphs: Uplifting any gnn with local structure awareness," *arXiv preprint arXiv:2110.03753*, 2021.

[35] K. Zhao, J. X. Yu, Q. Li, H. Zhang, and Y. Rong, "Learned sketch for subgraph counting: a holistic approach," *VLDB J.*, vol. 32, no. 5, pp. 937–962, 2023.

[36] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?" in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems, NeurIPS, December 6-12, virtual*, 2020.

[37] J. You, J. M. G. Selman, R. Ying, and J. Leskovec, "Identity-aware graph neural networks," in *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI , Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI, Virtual Event, February 2-9*. AAAI Press, 2021, pp. 10 737–10 745.

[38] Z. Yan, J. Zhou, L. Gao, Z. Tang, and M. Zhang, "An efficient subgraph GNN with provable substructure counting power," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD, Barcelona, Spain, August 25-29*. ACM, 2024, pp. 3702–3713.

[39] H. Zhang, J. X. Yu, Y. Zhang, K. Zhao, and H. Cheng, "Distributed subgraph counting: A general approach," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 2493–2507, 2020.

[40] Q. Li and J. X. Yu, "Fast local subgraph counting," *Proc. VLDB Endow.*, vol. 17, no. 8, pp. 1967–1980, 2024.

[41] K. Zhao, J. X. Yu, Z. He, and Y. Rong, "Learning with small data: Subgraph counting queries," in *Database Systems for Advanced Applications - 28th International Conference, DASFAA, Tianjin, China, April 17-20, Proceedings, Part III*, ser. Lecture Notes in Computer Science, vol. 13945. Springer, 2023, pp. 308–319.

[42] X. Yu, Z. Liu, Y. Fang, and X. Zhang, "Learning to count isomorphisms with graph neural networks," in *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI , Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI, Washington, DC, USA, February 7-14*. AAAI Press, 2023, pp. 4845–4853.

[43] F. Frasca, B. Bevilacqua, M. M. Bronstein, and H. Maron, "Understanding and extending subgraph gnns by rethinking their symmetries," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems, NeurIPS, New Orleans, LA, USA, November 28 - December 9*, 2022.

[44] E. Alsentzer, S. G. Finlayson, M. M. Li, and M. Zitnik, "Subgraph neural networks," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems, NeurIPS, December 6-12, virtual*, 2020.

[45] B. Tahmasebi, D. Lim, and S. Jegelka, "The power of recursion in graph neural networks for counting substructures," in *International Conference on Artificial Intelligence and Statistics, 25-27 April, Palau de Congressos, Valencia, Spain*, ser. Proceedings of Machine Learning Research, vol. 206. PMLR, 2023, pp. 11 023–11 042.

[46] L. Zhao, W. Jin, L. Akoglu, and N. Shah, "From stars to subgraphs: Uplifting any GNN with local structure awareness," in *The Tenth International Conference on Learning Representations, ICLR, Virtual Event, April 25-29*. OpenReview.net, 2022.

[47] H. Maron, H. Ben-Hamu, H. Serviansky, and Y. Lipman, "Provably powerful graph networks," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems, NeurIPS, December 8-14, Vancouver, BC, Canada*, 2019, pp. 2153–2164.

[48] Y. Huang, X. Peng, J. Ma, and M. Zhang, "Boosting the cycle counting power of graph neural networks with i$^2$-gnns," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=kDSmxOspsXQ

[49] T. Fu, C. Wei, Y. Wang, and R. Ying, "Desco: Towards generalizable and scalable deep subgraph counting," in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining, WSDM, Merida, Mexico, March 4-8*. ACM, 2024, pp. 218–227.

[50] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, "Tudataset: A collection of benchmark datasets for learning with graphs," in *ICML Workshop on Graph Representation Learning and Beyond*, 2020. [Online]. Available: www.graphlearning.io

[51] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *arXiv preprint arXiv:2005.00687*, 2020.

[52] https://anonymous.4open.science/r/MLSC-0302, 2025.

[53] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. correlation with molecular orbital energies and hydrophobicity," *Journal of Medicinal Chemistry*, vol. 34, no. 2, pp. 786–797, 1991. [Online]. Available: https://doi.org/10.1021/jm00106a046

[54] https://bitbucket.org/seshadhri/escape, 2017.

[55] https://bitbucket.org/nojan-p/orbit-counting/, 2020.

[56] https://github.com/magic62442/subgraph-counting, 2022.

[57] https://gitlab.com/steven3k/motivo/, 2019.

[58] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *7th International Conference on Learning Representations, ICLR, New Orleans, LA, USA, May 6-9*. OpenReview.net, 2019.

[59] https://github.com/weihua916/powerful-gnns, 2019.

[60] https://github.com/hadarser/ProvablyPowerfulGraphNetworks_torch, 2022.

[61] https://github.com/LingxiaoShawn/GNNAsKernel, 2022.

[62] https://github.com/GraphPKU/I2GNN, 2022.

[63] http://snap.stanford.edu/idgnn, 2022.

[64] https://github.com/pkuyzy/esc-gnn, 2024.

[65] https://github.com/fuvty/DeSCo, 2024.